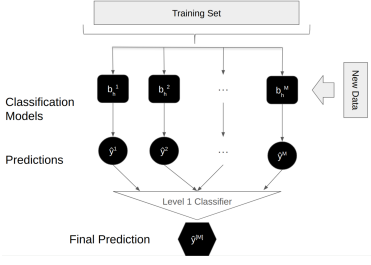
Applied Machine Learning

Ensembling: Stacking



Learning goals

- From Averaging to Stacking
- CV Stacking and Meta-Features
- Practical Guidance and AutoGluon

FROM AVERAGING TO STACKING

So far: weighted model averaging uses a **fixed** aggregation function G:

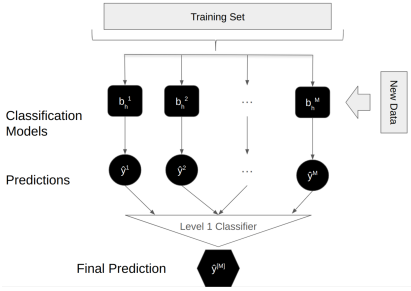
$$f_{\mathbf{w}}(\mathbf{x}) = \sum_{m=1}^{M} w_m \cdot b^{[m]}(\mathbf{x})$$

Key limitation: weights w_m are **global** — every input $\mathbf{x}$ gets the same weighting of models.

What if model 1 is better in one region of the input space, and model 2 in another?
$\rightarrow$ **Idea:** Replace the fixed G with a **learned model** that can adapt the combination to the input. This is **stacking**.

STACKING

Also called stacked generalization (▸ “Wolpert” 1992): train multiple base models (Level 0) on the same data and use their predictions as input features for a Level 1 model that makes the final ensemble prediction → heterogeneous ensembling



Steps:

1. Base model training (Level 0)
2. Collect predictions
3. Level 1 model training

In contrast to model averaging, stacking already combines predictions during training.

STACKING

Algorithm: Single Layer Stacking

Require: Training data $\mathcal{D} = \left(\left(\mathbf{x}^{(1)}, y^{(1)}\right), \ldots, \left(\mathbf{x}^{(n)}, y^{(n)}\right)\right)$

Ensure: Stacked Ensemble $f^{[M]}$

Step 1: Learn Level 0 base models
for $m \leftarrow 1$ to M **do**
  Fit base model $b^{[m]}$ on $\mathcal{D}$
end for

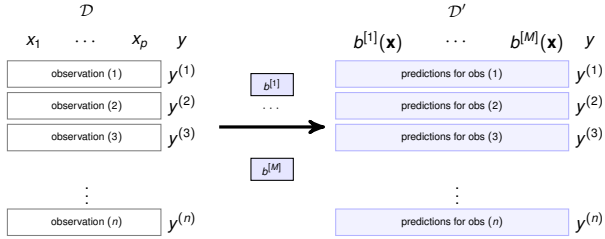
Step 2: Construct new dataset $\mathcal{D}'$ from $\mathcal{D}$
for $i \leftarrow 1$ to n **do**
  Add $(\mathbf{x}'^{(i)}, y^{(i)})$ to new dataset, where $\mathbf{x}'^{(i)} = (b^{[1]}(\mathbf{x}^{(i)}), \ldots, b^{[M]}(\mathbf{x}^{(i)}))$
end for

Step 3: Fit Level 1 model $f^{[M]}$ on $\mathcal{D}'$
return $f^{[M]}, b^{[1]}, \ldots, b^{[M]}$

NB: We can either drop the original features or keep them when learning the Level 1 model.

STACKING: DATA TRANSFORMATION

The key idea: base model predictions on the training data form a **new feature matrix** $\mathcal{D}'$ for the Level 1 model.



Each column m in $\mathcal{D}'$ contains model $b^{[m]}$'s predictions for all n training observations.
The labels y are carried over unchanged. The Level 1 model is then trained on $\mathcal{D}'$.

EXAMPLE: META-FEATURE MATRIX

Continuing the sonar example from the previous lecture ($M = 3$ base models: RF, k-NN, LogReg).
After collecting base model predictions, the meta-feature matrix $\mathcal{D}'$ might look like:

Obs	**RF** $\hat{\pi}^{[1]}$	**k-NN** $\hat{\pi}^{[2]}$	**LogReg** $\hat{\pi}^{[3]}$	y
1	0.82	0.91	0.65	1
2	0.35	0.28	0.41	0
3	0.71	0.45	0.73	1
4	0.12	0.33	0.22	0
5	0.58	0.80	0.49	1
⋮	⋮	⋮	⋮	⋮

- Each column = one base model's probability predictions (here: $P(y = 1|\mathbf{x})$)
- Each row = one training observation
- The Level 1 model (e.g., logistic regression) is trained on this $n \times 3$ matrix to predict y
- The meta-model can learn, e.g., "trust RF when it is confident, but rely on k-NN otherwise"

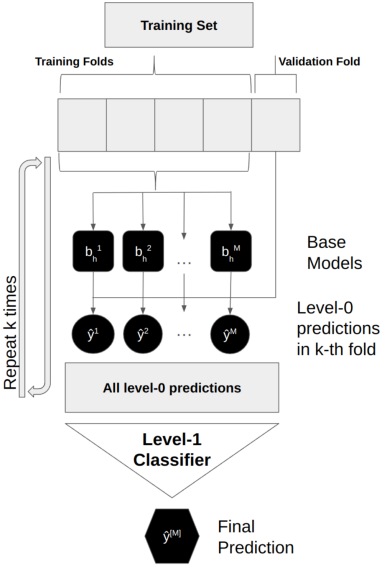
WHY NOT USE TRAINING PREDICTIONS DIRECTLY?

Problem: If base models predict on the same data they trained on, the predictions can be overly optimistic.

Example: A 1-nearest-neighbor classifier achieves 0% error on its training data, but may have 30% error on unseen data. $\rightarrow$ The meta-model would see 0% error and learn to fully trust this model.

Solution: Use **cross-validated (out-of-fold) predictions** to construct the meta-feature matrix. Each base model predicts only on data it was *not* trained on $\rightarrow$ predictions reflect true generalization quality.

STACKING CROSS-VALIDATED



Why use cross-validated predictions?

- **Avoids information leakage:** without CV, base models predict on their own training data → overconfident → meta-model overfits
- **Realistic error signal:** OOF predictions reflect true model quality
- **Data-efficient:** all n observations get a prediction → full-size meta-dataset

Blending vs. **Stacking:**

- Blending: single hold-out split (simpler, wastes data)
- Stacking: K-fold CV (data-efficient, more complex)

STACKING CROSS-VALIDATED

Algorithm: Single Layer Stacking with Cross-Validation

Require: Training data $\mathcal{D} = \left(\left(\mathbf{x}^{(1)}, y^{(1)} \right), \ldots, \left(\mathbf{x}^{(n)}, y^{(n)} \right) \right)$, number of folds K

Ensure: Stacked Ensemble $f^{[M]}$

for $k \leftarrow 1$ to K **do**

Step 1: Learn fold-specific Level 0 base models

for $m \leftarrow 1$ to M **do**

Fit base model $b_k^{[m]}$ on $\mathcal{D}_{\text{train}}^{(k)}$

end for

Step 2: Add out-of-fold predictions to new dataset $\mathcal{D}'$

for all $(\mathbf{x}^{(i)}, y^{(i)}) \in \mathcal{D}_{\text{test}}^{(k)}$ **do**

Add $(\mathbf{x}'^{(i)}, y^{(i)})$ to new dataset, where $\mathbf{x}'^{(i)} = (b_k^{[1]}(\mathbf{x}^{(i)}), \ldots, b_k^{[M]}(\mathbf{x}^{(i)}))$

end for

end for

Step 3: Fit Level 1 model $f^{[M]}$ on $\mathcal{D}'$

return $f^{[M]}$

NB: For prediction, the base models must be retrained on the full dataset $\mathcal{D}$ (see Forward Pass).

CV STACKING: HOW THE META-FEATURE MATRIX IS BUILT

Example: $K = 3$ folds, $M = 2$ base models, $n = 9$ observations. Each base model produces one OOF prediction per observation.

Training Data $\mathcal{D}$

Obs	Fold
1	1
2	1
3	1
4	2
5	2
6	2
7	3
8	3
9	3

OOF →

Meta-Feature Matrix $\mathcal{D}'$

Obs	$b^{[1]}$	$b^{[2]}$	y
1	pred	pred	$y^{(1)}$
2	pred	pred	$y^{(2)}$
3	pred	pred	$y^{(3)}$
4	pred	pred	$y^{(4)}$
5	pred	pred	$y^{(5)}$
6	pred	pred	$y^{(6)}$
7	pred	pred	$y^{(7)}$
8	pred	pred	$y^{(8)}$
9	pred	pred	$y^{(9)}$

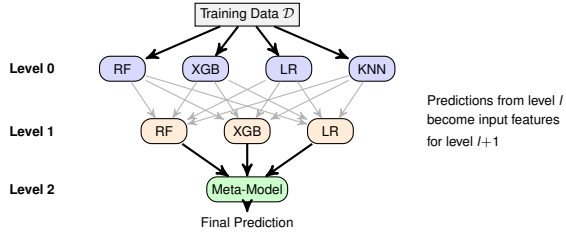
Colors = fold assignment.

Each prediction is out-of-fold: model trained on other folds, predicts held-out fold.

$\Rightarrow$ Every observation gets exactly one OOF prediction per base model.

MULTILEVEL STACKING

So far: train Level 0 base models, use predictions to train the Level 1 model. Now: Also train multiple models at Level 1, use predictions to train final Level 2 model.



- In principle: repeat at any level, creating deep stacked ensembles. In practice, 2–3 layers suffice.
- Looks like a neural network, but: each layer is trained **independently** (no backpropagation), and layers use **heterogeneous** model types rather than uniform neurons.

HIGHER LEVEL MODELS: META-LEARNER CHOICE

Popular choices of Level 1 or Level 2 models:

- **Linear** / **logistic regression** (most common)
  - Simple meta-model reduces overfitting risk on the small meta-feature space (M columns)
  - Non-negative weight constraints can improve stability
- **GES:** greedily selects and averages predictions of the models at the previous level

NB: A powerful meta-model (e.g., gradient boosting) can overfit the meta-features, especially with few base models. Simpler is often better here.

HIGHER LEVEL MODELS: META-FEATURE DESIGN

Probabilities vs. hard labels as meta-features (▸ "Ting and Witten" 1999):

- Hard labels destroy confidence information: predictions 0.51 and 0.99 both map to class 1, but mean very different things
- Probability predictions preserve this signal $\rightarrow$ meta-model can learn *when* a base model is confident
- $\Rightarrow$ Always prefer probability outputs as meta-features for classification

Include original features X alongside meta-features?

- Can help when base models miss informative features (e.g., a feature no base model uses)
- Increases overfitting risk: the meta-model now has $p + M$ features instead of M
- In practice: try both, evaluate via nested CV

FORWARD PASS / PREDICTION

Assume: trained base models $b^{[1]}, \ldots, b^{[M]}$ and meta-model $f^{[M]}$

Require: Test data $\mathbf{X}_{\text{test}} = (\mathbf{x}^{(1)}, \ldots, \mathbf{x}^{(n_{\text{test}})})$

Ensure: Predictions $\hat{\mathbf{y}}_{\text{test}}$

Step 1: Predict with each base model

for $m \leftarrow 1$ to M **do**

$\hat{\mathbf{y}}^{[m]} \leftarrow b^{[m]}(\mathbf{X}_{\text{test}})$

end for

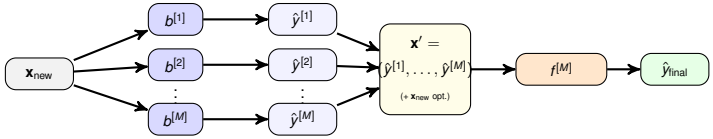
Step 2: Construct meta-feature matrix

$\mathbf{X}' \leftarrow [\hat{\mathbf{y}}^{[1]}, \ldots, \hat{\mathbf{y}}^{[M]}]$ or $\mathbf{X}' \leftarrow [\mathbf{X}_{\text{test}}, \hat{\mathbf{y}}^{[1]}, \ldots, \hat{\mathbf{y}}^{[M]}]$ (if original features are retained)

Step 3: Predict with the meta-model

return $f^{[M]}(\mathbf{X}')$

FORWARD PASS: VISUAL OVERVIEW



At prediction time, no cross-validation is needed: simply run all base models (trained on full data), assemble meta-features, and feed them to the trained meta-model.

CONCEPTUAL CHECK

What other methods do the following level-1 classifiers recover?

- Fit the level-1 training data with a linear model that only depends on one feature, and that feature is chosen such that ρ, our metrics of interest, is optimized.
- The level-1 model is a linear model that assigns equal weight $w_m = \frac{1}{M}$ to every input feature.

CONCEPTUAL CHECK – ANSWERS

- A linear model that selects the single best input feature (= base model prediction) and optimizes ρ is equivalent to **model selection**: simply pick the best-performing base model.
- A linear model with equal weights $w_m = \frac{1}{M}$ for every input feature is equivalent to **simple model averaging**: $\frac{1}{M} \sum_{m=1}^{M} b^{[m]}(\mathbf{x})$.

$\rightarrow$ Stacking with a flexible level-1 model generalizes both model selection and model averaging!

WHEN DOES STACKING HELP?

Stacking tends to **help** when:

- Base models are diverse (different algorithms, different hyperparameters)
- Base models have complementary strengths (good in different regions of input space)
- Training set is large enough for reliable out-of-fold predictions

Stacking can **hurt** when:

- Base models are too similar $\rightarrow$ meta-model has nothing useful to learn
- Dataset is small $\rightarrow$ out-of-fold predictions are noisy, meta-model overfits
- Only one base model clearly dominates $\rightarrow$ model selection is simpler and equally effective

NB: Simple model averaging is a surprisingly strong baseline; stacking is not always worth the added complexity.

PRACTICAL TIPS FOR STACKING

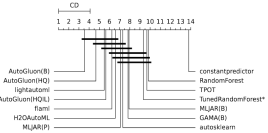
- **Number of folds:** $K = 5$ or $K = 10$ is typical for CV stacking. Smaller $K \rightarrow$ more bias in OOF predictions; larger $K \rightarrow$ more computation.
- **Number of base models:** Diminishing returns after $\sim$5–10 diverse models. Focus on diversity over quantity.
- **Tune first, then stack:** Treat hyperparameter tuning of base models and stacking as two separate stages. Do not tune base models *inside* the stacking CV loop.
- **Avoid double-dipping:** The OOF predictions used to train the meta-model should *not* also be used to select which base models to include $\rightarrow$ use nested CV for an honest estimate of the stacked ensemble's performance.
- **Start simple:** Try equal-weight averaging first. Only move to stacking if averaging leaves clear room for improvement.

AUTOGLUON

▸ “Erickson et al.” 2020

AutoGluon

https://auto.gluon.ai/



AutoGluon on AMLB 1h binary.

- De-facto most popular and successful AutoML system (Amazon)
- Also supports multimodal and time-series data
- Does not use HPO; instead uses default configurations per learner
- Employs multi-layer stacking (typically 2–3 layers)
  - Same model types repeated in each layer
  - **Repeated k-fold bagging:** reduces OOF prediction variance
  - **Feature passthrough:** original features at every layer
- Uses GES to select and average predictions of the last layer

SUMMARY

- **Stacking** replaces a fixed aggregation function with a **learned meta-model** trained on base model predictions
- Use **cross-validated (out-of-fold) predictions** to avoid information leakage when constructing the meta-feature matrix
- After CV stacking, **retrain** base models on full data for deployment
- Keep the meta-model **simple** (e.g., linear/logistic regression) and use **probability** outputs as meta-features
- Stacking generalizes both model selection and model averaging
- Multi-layer stacking (e.g., AutoGluon) extends this to multiple levels with repeated bagging and feature passthrough